



Pattern Mining in R: Association Rules, Itemsets, and Sequences



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Navigating Pattern Mining

Four progressively different questions about the same transactional data. A shared mechanic runs through all of them: when supp, conf, or support are set to 0, the miner estimates the threshold during fit() through explicit strategies.

1. Association Rules

Directional patterns with a left-hand side implying a right-hand side.

2. Itemsets

Undirected co-occurrence structures, filtered by which items may appear.

3. Sequential Patterns

Recurring structures where order matters, not just co-occurrence.

4. Interesting Rule Filters

Post-mining filters that decide which discovered rules are worth keeping.

Association Rules – pat_apriori

Directional rules with a fixed consequent (rhs) and explicit filtering by confidence and lift — the entry point for the whole pattern-mining family.

```
pm <- pat_apriori(
  target = "rules", supp = 0, conf = 0,
  support_strategy =
    pat_support_threshold("curvature"),
  confidence_strategy =
    pat_confidence_threshold(
      "rhs_baseline", margin = 0.1),
  minlen = 2, maxlen = 3,
  rhs = c("native-country=US"),
  quality_filter = patutils()$quality_min(
    confidence = 0.9, lift = 1.03)
)
pm <- fit(pm, trans)
rules <- discover(pm, trans)
```

- **Baseline-relative confidence:** a fixed rhs lets `pat_confidence_threshold("rhs_baseline")` set confidence relative to how frequent the consequent already is.
- **A second filter:** lift acts on top of support/confidence — it removes rules that are frequent but not informative.

Itemsets – pat_eclat

Frequent itemset mining: groups of items that often appear together, without any directional implication.

```
pm <- pat_eclat(  
  supp = 0,  
  support_strategy =  
    pat_support_threshold("curvature"),  
  maxlen = 3,  
  include = c("sex=Male",  
    "marital-status=Married-civ-spouse"),  
  exclude = c("income=small"),  
  quality_filter =  
    patutils()$quality_min(support = 0.4)  
)  
pm <- fit(pm, trans)  
itemsets <- discover(pm, trans)
```

- **Structural filters first:** include/exclude act as semantic filters over which items may appear — applied before the quality filter.
- **Co-occurrence, not implication:** itemsets capture co-occurrence — the key conceptual difference from association rules.

Sequential Patterns – pat_cspade

The same pattern-mining workflow, but the data type shifts from itemsets to ordered event sequences — the question becomes "what recurring order of events happens," not "what items co-occur."

```
pm <- pat_cspade(  
  support = 0,  
  support_strategy = pat_support_threshold(  
    "min_count", min_count = 4),  
  maxlen = 3,  
  quality_filter =  
    patutils()$quality_min(support = 0.5)  
)  
pm <- fit(pm, sequence_data)  
seqs <- discover(pm, sequence_data)
```

- **Different input shape:** sequence-transaction data — each case is an ordered basket of events, not a flat transaction table.
- **Same rhythm:** otherwise unchanged from pat_apriori/pat_eclat — same configure/fit/discover/evaluate rhythm.

Interesting Rule Filters

Discovery and interestingness filtering are separate concerns: a miner generates candidate rules, and a filter decides which ones remain.

```
dara_filter <- pat_rule_filter_dara(  
  data, rhs_attr = "income",  
  rhs_value = "small", min_score = 1)  
dara_rules <- pat_filter_rules(  
  rules, dara_filter)  
attr(dara_rules, "dara_ranking")  
[["marital-status"]]
```

- **pat_rule_filter_none()**: the explicit no-filter default — keeps every candidate rule.
- **pat_rule_filter_interest()**: classical measures from support relationships (lift, Kulczynski, imbalance ratio).
- **pat_rule_filter_dara()**: scores each antecedent by how much more prominent it becomes in the rule set than in the target subset.
- **pat_filter_rules()**: applies any filter to a rule table or an arules object; a rule_filter can also be passed directly into pat_apriori().

What to Observe

- **One shared experiment shape:** all four techniques share configure the miner → fit() → discover() → evaluate().
- **Design decisions live in thresholds and filters:** supp/conf strategies control the search space, include/exclude control the vocabulary, and interest filters control what survives after mining.



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br

<https://eic.cefet-rj.br/dal>